

Complete Pattern Notes
Self-Study & Last-Minute Revision Guide

DSA Stack & Queue

Stack & Queue Patterns — Overview

Pattern	Core Idea	Signal Words
Monotonic Deque	Deque maintaining order, front=max/min of current window	sliding window max/min, window of size k
Expression Evaluation	Stack defers lower-precedence operations, handles nesting via save/restore	calculate, evaluate expression, parentheses
Stack Simulation	Current element only interacts with most recent element (LIFO)	collision, decode, remove adjacent duplicates
BFS Hard Variants	Multi-source: all sources at level 0. 0-1 BFS: deque with cost-based insertion	spread simultaneously, 0 or 1 cost edges

Stack and Queue appear in Trees (level-order BFS), Graphs (topological sort, Dijkstra), and Heap topics too — those are covered in their respective sections.

Pattern 1 — Monotonic Deque

Core Idea: A deque maintaining decreasing (or increasing) order. Front always holds the max (or min) of the current window. Use when sliding window needs max/min queries in $O(1)$.

Monotonic Stack vs Monotonic Deque

	Monotonic Stack	Monotonic Deque
Ends used	One end only	Both ends (front + back)
Used for	Next/prev greater/smaller	Sliding window max/min
Window	No fixed window	Fixed window of size k
Remove old	Not needed	Pop from front when index expired
Time	$O(n)$	$O(n)$

Template — Sliding Window Maximum

```
deque<int> dq;    // stores INDICES not values
for(int i=0; i<n; i++){
    // remove expired elements from front
    if(!dq.empty() && dq.front() == i-k)
        dq.pop_front();

    // maintain decreasing order from back
    while(!dq.empty() && nums[dq.back()] < nums[i])
        dq.pop_back();

    dq.push_back(i);

    // record answer when window is full
    if(i >= k-1)
        res.push_back(nums[dq.front()]); // front = max
}
```

Why store indices not values: you need to check if front element has expired ($dq.front() == i-k$). Can't do this with values alone.

Dry Run — $nums=[1,3,-1,-3,5,3,6,7]$, $k=3$

```
i=0: dq=[0]
i=1: 3>1 -> pop 0, push 1 -> dq=[1]
i=2: -1<3 -> push 2 -> dq=[1,2] window full -> max=nums[1]=3
i=3: -3<-1 -> push 3 -> dq=[1,2,3] max=nums[1]=3
i=4: 5>all -> pop 3,2,1 -> dq=[4] max=nums[4]=5
i=5: 3<5 -> push 5 -> dq=[4,5] max=nums[4]=5
i=6: 6>3,5 -> pop 5,4 -> dq=[6] max=nums[6]=6
i=7: 7>6 -> pop 6 -> dq=[7] max=nums[7]=7
```

```
result = [3,3,5,5,6,7]
```

Solved Problems

Problem	LC#	Key Insight
Sliding Window Maximum	239	Decreasing deque, front=max. Pop back when new element larger, pop front when expired.

Pattern 2 — Expression Evaluation

Core Idea: Stack defers lower-precedence operations. Higher precedence ($$ / $/$) execute immediately. Parentheses save/restore context at each nesting level.*

The 3 Problem Types

Type	Operators	Key Challenge	Leetcode
Basic (no parens)	+ - only	No precedence issues, just track sign	No LC
With * and /	+ - * /	Precedence: execute $*$ / $/$ immediately	227
With parentheses	+ - and ()	Save/restore context at each (and)	224

Leetcode 227 — Basic Calculator II (no parentheses)

Key insight: + and - are deferred (pushed to stack with sign). * and / execute immediately using stack top.

Final answer = sum of stack.

```
int num=0; char op='+'; // treat first number as +num
for(int i=0; i<s.size(); i++){
    if(isdigit(s[i])) num = num*10 + (s[i]-'0');
    if((!isdigit(s[i]) && s[i]!=' ') || i==s.size()-1){
        if(op=='+') st.push(num);
        if(op=='-') st.push(-num);
        if(op=='*'){ int t=st.top(); st.pop(); st.push(t*num); }
        if(op=='/'){ int t=st.top(); st.pop(); st.push(t/num); }
        op=s[i]; num=0;
    }
}
```

```
// sum all stack elements for final answer
```

Why start with op='+': first number has no operator before it — treating it as +num pushes it correctly to stack.

Leetcode 224 — Basic Calculator (with parentheses)

Key insight: At (— save current result and current sign, reset both for fresh inner computation. At) — restore saved sign and result, combine with inner result.

```
stack<int> st;
int num=0, result=0, op=1; // op=1 means +, op=-1 means -
for(char c : s){
    if(isdigit(c)){ num=num*10+(c-'0'); }
    else if(c=='+'){ result+=num*op; op=1; num=0; }
    else if(c=='-'){ result+=num*op; op=-1; num=0; }
    else if(c=='('){
        st.push(result); // save outer result
        st.push(op); // save sign before (
```

```

        result=0; op=1;    // fresh start inside
    }
    else if(c=='-'){
        result += num*op; num=0;
        result *= st.top(); st.pop();    // apply saved sign
        result += st.top(); st.pop();    // add saved outer result
    }
}
return result + num*op;    // flush last number

```

What Gets Pushed at (and Popped at)

Event	Action	Why
See (	Push current result, push current sign, reset both	Save outer context, start fresh inside
See)	Flush num, pop sign (multiply), pop result (add)	Apply saved sign to inner result, combine with outer
End	return result + num*op	Last number never flushed by an operator

When to use two stacks vs one: if you're saving two different TYPES (string + int like in Decode String) use two stacks. If both are int (result + sign like in 224), use one stack pushing both as ints.

Solved Problems

Problem	LC#	Key Insight
Basic Calculator II	227	op='+' start. + and - push to stack. * and / execute immediately with stack top.
Basic Calculator	224	At (push result+sign, reset. At) flush num, pop sign, pop result, combine.

Pattern 3 — Stack Simulation

Core Idea: Problems where the current element ONLY interacts with the most recently seen element. That LIFO (last in, first out) nature maps directly to a stack.

The Signal Question

Ask yourself: Does the current element only care about what came immediately before it? If yes → stack simulation.

Type 1 — Collision Problems

Asteroid Collision — Leetcode 735

Array of integers = asteroids. Positive = moving right, negative = moving left. When two meet — smaller explodes. Equal = both explode. Find final state.

```
Collision only when: stack top > 0 AND current < 0 (moving toward each other)
for each asteroid a:
    while stack not empty AND top > 0 AND a < 0:
        if top < -a -> top explodes, pop, continue checking
        if top == -a -> both explode, pop, mark a destroyed, break
        if top > -a -> a explodes, mark a destroyed, break
    if a not destroyed -> push a
```

No collision cases: positive+positive (same direction), negative+negative (same direction), negative+positive (moving away from each other).

Type 2 — Decode/Expand Problems

Decode String — Leetcode 394

Given encoded string like '3[a2[c]]' — decode it. k[string] means repeat string k times. Brackets can be nested.

```
Two stacks: one for counts (int), one for strings (string)
at '[': push current count to count_stack
        push current string to string_stack
        reset both to 0 and empty string
at ']': repeat current string by count_stack.top()
        prepend string_stack.top() to result
        pop both stacks
digits: build multi-digit number
letters: append to current string
```

Critical rule: ALWAYS push current string at '[' even if empty. String stack and count stack must stay in sync — one push per '[', one pop per ']'.

When to use two stacks: When you need to save two different types at each nesting level. Decode String saves string + int → two stacks. Basic Calculator saves int + int → one stack is enough.

Type 3 — Remove/Clean Problems

Remove All Adjacent Duplicates — Leetcode 1047

Repeatedly remove adjacent duplicate characters. Each new character only needs to check the immediately previous surviving character.

```
for each char c:
    if stack not empty AND stack.top() == c -> pop (both removed)
    else -> push c
result = reverse of stack contents
```

Stack Simulation Signal Words

Signal	Type
collision, survive, destroy, explode	Collision stack
decode, expand, repeat k times	Decode stack (two stacks for different types)
remove adjacent duplicates, backspace	Clean stack
undo operation, most recent first	General LIFO simulation

Solved Problems

Problem	LC#	Key Insight
Asteroid Collision	735	Collision only when top>0 and current<0. Three outcomes: top explodes, current explodes, both explode.
Decode String	394	Two stacks. Always push at '[' even if empty. Pop and combine at ']'.].
Remove All Adjacent Duplicates	1047	Single stack. Pop if top matches current, else push. Reverse at end.

Pattern 4 — BFS Hard Variants

Core Idea: BFS finds shortest path in unweighted graphs. Hard variants extend this: multiple simultaneous sources (multi-source BFS) or edges with 0/1 cost (0-1 BFS).

Standard BFS vs Hard Variants

Type	When to Use	Data Structure	Time
Standard BFS	Single source, unweighted	Queue	$O(V+E)$
Multi-source BFS	Multiple sources, min dist from any	Queue (all sources at level 0)	$O(V+E)$
0-1 BFS	Edge weights 0 or 1 only	Deque	$O(V+E)$
Dijkstra	Any non-negative weights	Min Heap	$O((V+E) \log V)$

Multi-Source BFS

Key insight: Add ALL sources to the queue simultaneously at level 0. BFS naturally computes minimum distance from the nearest source for every cell.

Why not run BFS from each source separately: That's $O(k*n)$. Multi-source is $O(n)$ — all sources spread in parallel.

```
queue<pair<int,int>> q;
// Step 1: add ALL sources at level 0
for(all source cells)
    q.push(source); visited[source]=true;
// Step 2: standard BFS
while(!q.empty()){
    int size=q.size();
    while(size--){
        auto [r,c]=q.front(); q.pop();
        for(each neighbor nr,nc){
            if(valid && !visited[nr][nc]){
                visited[nr][nc]=true;
                dist[nr][nc]=dist[r][c]+1;
                q.push({nr,nc});
            }
        }
    }
    minutes++; // each level = 1 time unit
}
```

Process level by level (tracking size before each level) when you need to count time/steps. Each level = 1 time unit of simultaneous spread.

Rotting Oranges — Leetcode 994

All rotten oranges spread simultaneously. Add all initially rotten to queue at level 0. Count fresh oranges. BFS spreads rotten — decrement fresh count. If fresh==0 at end, return minutes. Else return -1.

```
// count fresh and add all rotten to queue
for each cell: if rotten -> q.push({r,c}), if fresh -> fresh++
while(!q.empty() && fresh>0){
    process entire current level (one minute)
    for each rotten cell: spread to adjacent fresh cells
        fresh--, mark as rotten, push to queue
    minutes++
}
return fresh==0 ? minutes : -1
```

0-1 BFS

Key insight: Weight-0 edges are 'free moves' — push to FRONT of deque (process immediately, same distance). Weight-1 edges cost something — push to BACK (process later, higher distance).

This maintains correct distance ordering without a full priority queue → $O(V+E)$ instead of $O((V+E) \log V)$.

```
deque<pair<int,int>> dq; // {distance, node}
dq.push_back({0, start});
dist[start]=0;
while(!dq.empty()){
    auto [d, node]=dq.front(); dq.pop_front();
    for(each neighbor with edge weight w){
        if(d+w < dist[neighbor]){
            dist[neighbor]=d+w;
            if(w==0) dq.push_front({d, neighbor}); // free -> front
            else     dq.push_back ({d+1, neighbor}); // cost -> back
        }
    }
}
```

0-1 BFS vs Dijkstra: use 0-1 BFS when edge weights are strictly 0 or 1. It's $O(V+E)$ — faster than Dijkstra's $O((V+E) \log V)$. For general non-negative weights, use Dijkstra.

BFS Signal Words

Signal	BFS Type
shortest path in unweighted graph/grid	Standard BFS
all sources spread simultaneously	Multi-source BFS
minimum distance from any source	Multi-source BFS
0 or 1 cost edges, minimum cost path	0-1 BFS
level by level traversal, time steps	BFS with level tracking

Solved Problems

Problem	LC#	Key Insight
Rotting Oranges	994	Multi-source BFS. All rotten at level 0. Count fresh, decrement on spread. Return -1 if any fresh remain.
Min Cost Valid Path in Grid	1368	0-1 BFS. Moving in given direction=cost 0 (front). Changing direction=cost 1 (back).

Quick Reference — All 4 Stack & Queue Patterns

Pattern	Data Structure	Time	Use When
Monotonic Deque	Deque	$O(n)$	Sliding window max/min of size k
Expression Evaluation	Stack	$O(n)$	Calculate expressions with operators and parentheses
Stack Simulation	Stack (1 or 2)	$O(n)$	Current element interacts only with most recent (LIFO)
Multi-source BFS	Queue	$O(V+E)$	Min distance from any of multiple sources simultaneously
0-1 BFS	Deque	$O(V+E)$	Shortest path with edge weights 0 or 1 only

Most Important Rules

- Monotonic Deque: store indices not values. Pop back to maintain order. Pop front when expired ($\text{index} == i-k$).
- Expression 227: start with $\text{op}='+'$. Push signed value for $+$ and $-$. Execute $*$ and $/$ immediately with stack top.
- Expression 224: at $($ push result then sign, reset both. At $)$ flush num, pop sign (multiply), pop result (add).
- Stack Simulation: ask 'does current element only care about the most recent one?' If yes $\rightarrow$ stack.
- Two stacks when: saving two different types at same nesting level (string + int). One stack when: same type (int + int).
- Multi-source BFS: add ALL sources at level 0 simultaneously. Process level by level for time counting.
- 0-1 BFS: weight 0 $\rightarrow$ push_front (free, same distance). Weight 1 $\rightarrow$ push_back (costs, higher distance).
- BFS patterns in Trees (level-order), Graphs (Dijkstra, topological sort), and Heap are covered in those topics.

— End of Notes —